SEMINÁRIO DEVOPS · TEMA T02

Docker Compose

API e banco rodando juntos, com os dados guardados em um volume

```
$ docker compose up -d
✓ Container redis  Started
✓ Container api    Started
```

Equipe Galegos · UNINASSAU · DevOps · 2026.2

Ítalo Waldecy Pereira dos Santos · Hugo Duarte Dias · Caio Gomes Ferrão D’avila · Caetano Mesquita · Helio Gomes Cunha

ROTEIRO

30 minutos, cinco blocos, uma demonstração ao vivo

Bloco	Tempo	Quem fala
1. Problema e caso de uso	3 min	Ítalo
2. Fundamentos e decisões	6 min	Hugo e Caio
3. Arquitetura e fluxo	4 min	Caio e Caetano
4. Demonstração ao vivo	12 min	Caetano e Helio
5. Limitações, conclusão e fontes	5 min	Ítalo e Hugo
Perguntas	8 min	Todos

BLOCO 1 · PROBLEMA

Subir uma aplicação com banco “na mão” dá trabalho

```
docker network create rede
docker volume create dados
docker build -t api .
docker run -d --name redis
  --network rede -v dados:/data redis:7-alpine
docker run -d --name api
  --network rede -p 5000:5000 api
```

A ordem importa: o banco precisa existir antes da API.

Tudo manual: rede, volume e variáveis criados um a um.

Comandos longos, fáceis de errar e de esquecer.

Cada pessoa monta de um jeito: difícil de repetir.

O Compose troca esse passo a passo por um arquivo e um comando.

BLOCO 1 · CASO DE USO

Nosso caso: um contador de visitas

API (Flask)

A cada acesso a **/**, soma 1 na chave **visitas** e devolve o total.

Banco (Redis)

Guarda o contador e grava em disco, dentro de um **volume**.

O que vamos provar

Se o container do Redis for **recriado**, o contador continua de onde parou.

```
$ curl.exe http://localhost:5000
A API foi acessada 3 vezes.
```

Imagem e container não são a mesma coisa

Imagem

- Modelo imutável, montado em camadas
- Criada a partir de um Dockerfile
- Uma imagem pode gerar vários containers

Container

- Instância de uma imagem em execução
- Ciclo de vida: criar, iniciar, parar, remover
- O que é gravado dentro dele some quando ele é removido

Dado importante não pode morar dentro do container.

Compose: um arquivo descreve a aplicação inteira

`services`

Os containers da aplicação. Aqui: **api** e **redis**.

`network`

Criada sozinha pelo Compose. Os serviços se acham **pelo nome**.

`volumes`

Armazenamento nomeado, **fora** dos containers.

```
$ docker compose up -d --build
$ docker compose down
```

Na rede do Compose, o nome do serviço vira o endereço

```
# docker-compose.yml
environment:
  REDIS_HOST: redis
# app.py
cache = redis.Redis(
  host="redis", port=6379)
```

Uma rede só da aplicação, criada pelo Compose.

DNS interno: “redis” aponta para o container do Redis.

Sem IP no código: se o container for recriado, o nome continua valendo.

Só a porta da API precisa ser publicada para o usuário.

Volume: os dados vivem fora do container

1

O container do Redis é **removido**.

2

O volume **meus-dados** continua existindo.

3

Um container **novo** monta o mesmo volume em **/data**.

4

O contador continua de onde parou.

```
command: redis-server
  --appendonly yes
volumes: meus-dados:/data
```

--appendonly yes: o Redis registra cada escrita em disco.

Volume nomeado é gerenciado pelo Docker; **bind mount** liga uma pasta do computador.

As decisões do projeto e o porquê de cada uma

Decisão	Motivo
Flask na API	Poucas linhas: o foco é o Compose, não a aplicação
Redis como banco	Chave-valor simples; INCR soma de forma atômica
Volume nomeado	Gerenciado pelo Docker e independente do caminho do computador
--appendonly yes	Grava cada escrita em disco, sem depender de snapshots
Imagens **alpine**	Menores: baixam e sobem mais rápido
.env.example	Configuração fora do código, com valores padrão que funcionam

BLOCO 3 · ARQUITETURA E FLUXO

Do navegador ao volume, passando pela rede do Compose



1
GET / chega à API na porta 5000

2
A API soma 1 em visitas no host redis

3
O Redis grava em /data, no volume

4
A API responde com o novo total

O docker-compose.yml, em poucas linhas

```yaml
services:
  api:
    build: .
    ports: ["5000:5000"]
    environment:
      REDIS_HOST: ${REDIS_HOST:-redis}
    depends_on: [redis]
  redis:
    image: redis:7-alpine
    command: ["redis-server", "--appendonly", "yes"]
    volumes:
      - meus-dados:/data
volumes:
  meus-dados:
```

build: .

A imagem da API sai do Dockerfile desta pasta.

depends_on

A API sobe depois do Redis: define a ordem, não a prontidão.

volumes

meus-dados é montado em /data dentro do Redis.

BLOCO 4 · DEMONSTRAÇÃO AO VIVO

Demonstração (1/3): subir e acessar

1	`docker compose version`	Registrar as versões usadas
2	`docker compose up -d --build`	Constrói a imagem da API e sobe os dois serviços
3	`docker compose ps`	api e redis em execução
4	`curl.exe http://localhost:5000`	Repetir 3 vezes: o contador sobe

No PowerShell, use **curl.exe**: sozinho, “curl” é um apelido de outro comando.

Demonstração (2/3): recriar o container do Redis

5	`docker compose rm -sf redis`	Remove o container do Redis. **Não acesse a API agora.**
6	`docker compose up -d redis`	Cria um container novo, do zero
7	`docker compose ps`	O Redis aparece como recém-criado
8	`curl.exe http://localhost:5000`	**O contador continua de onde parou**
9	`docker volume ls`	Aparece o volume que termina em meus-dados

BLOCO 4 · DEMONSTRAÇÃO AO VIVO

Demonstração (3/3): down e down -v

```
docker compose down
```

Remove os containers e a rede.

Mantém o volume: os dados continuam.

```
docker compose down -v
```

Remove também o volume.

Apaga os dados: o contador volta ao início.

```
$ docker compose down -v
$ docker compose up -d
$ curl.exe http://localhost:5000  # o contador recomeça
```

BLOCO 5 · LIMITAÇÕES

O que o nosso projeto ainda não resolve

Um computador só

O Compose roda em um host: não escala nem se recupera entre servidores.

Redis sem senha

Sem autenticação e com a porta 6379 exposta: só serve para demonstração local.

Volume não é backup

Se o disco do computador falhar, o volume vai junto.

Ordem, não prontidão

O depends_on não espera o Redis estar pronto para receber conexões.

Flask de desenvolvimento

O servidor embutido não é indicado para produção.

Redis não é relacional

Ótimo para contador e cache; não substitui um banco relacional.

BLOCO 5 · PRÓXIMOS PASSOS

Como evoluir o projeto

1. **Healthcheck no Redis** e depends_on com condition: service_healthy
2. **Fixar as versões** das dependências Python e das imagens
3. **.dockerignore** e usuário sem privilégios na imagem da API
4. **Não publicar a porta 6379** e proteger o Redis com senha
5. **Servidor de produção** para o Flask e testes automatizados

BLOCO 5 · CONCLUSÃO

O que levamos desta apresentação

1. Um arquivo descreve a aplicação inteira.
2. A rede liga os serviços pelo nome.
3. O volume separa a vida dos dados da vida do container.

Fontes

Docker Compose
docs.docker.com/compose

Redis: persistência em disco
redis.io/docs · Persistence

Flask
flask.palletsprojects.com

Código e README do projeto
github.com/Italosantoss03/projetoDevOps

Perguntas?

Obrigado!
Equipe Galegos